

A Tiny Story-Spec Property-Pattern DSL for Quest–Resource Story Graphs

1 Motivation

We consider automated verification over an *abstract story graph* whose state captures (i) quest/story nodes and traversal history, and (ii) resources such as items, flags, and skills. To express high-level design intents (progression gates, safety constraints, tutorial ordering, and local navigation guarantees), we propose a small, designer-facing DSL inspired by property-pattern languages used in runtime verification and cyber-physical systems (CPS) testing. These frameworks provide reusable, high-level templates that compile into LTL/MTL-style formulas and monitoring code.

2 Background: LTL and Bounded Temporal Operators

Linear Temporal Logic (LTL) is a formal language for describing how propositions should hold over a single sequence of states. LTL is commonly used in model checking to specify safety and liveness properties.

Let p and q be state propositions:

- $X p$ (neXt): p holds in the next state.
- $F p$ (Finally / Eventually): p holds at some future state.
- $G p$ (Globally / Always): p holds at all future states.
- $p U q$ (Until): p holds until q holds, and q must eventually hold.

When we need explicit bounds (e.g., “within k steps”), we use a bounded eventuality operator $F^{\leq k}$ as syntactic sugar. This can be interpreted as (i) an MTL-style bounded temporal operator over discrete steps, or (ii) a compilation target that is implemented via bounded graph search and introduced into LTL as a derived predicate (Section 8).

3 Execution Model

We assume a transition system whose states factor into:

1. **Story progression:** current story node, visited history, active plot labels or rooms.
2. **Resources/flags:** inventory, learned skills, tutorials seen, keys, booleans.
3. **Local navigation/action layer (optional):** a context-dependent graph used only by reachability rules.

The DSL references:

- **Events** emitted on transitions, e.g. `EnterRoom( $R$ )`, `EnterPlot( $X$ )`, `FirstEnterNode( $N$ )`.
- **State predicates** evaluated over resources/flags, e.g. `HasSeenNode( $T$ )`, `HasAnySkill $\{A, B\}$` .
- **Bounded reachability** in a navigation/action graph, e.g. `ReachObject( $O$ ) WITHIN  $k$  STEPS`.

4 DSL Sketch

4.1 Pseudo-Concrete Syntax

```
RULE BedReachableInRoomR:
  WHEN EnterRoom("InnUpperFloor")
  THEN ReachObject("Bed") WITHIN 20 STEPS

RULE TutorialBeforeBoss:
  WHEN FirstEnterNode("Boss_Guardian")
  REQUIRE HasSeenNode("StatusEffectTutorial")

RULE SkillAvailableForPlotX:
  WHEN EnterPlot("Plot_X")
  REQUIRE HasAnySkill { "Parry", "DodgeRoll" }
```

4.2 Additional Zelda-Flavored Examples

```
RULE BossKeyBeforeForestDoor:
  WHEN FirstEnterNode("ForestBossDoor")
  REQUIRE HasItem("BossKey_Forest")

RULE NoSoftlockWithoutWeaponInForest:
  WHEN EnterRoom("ForestTemple")
  REQUIRE HasAnyItem { "Sword", "DekuSticks", "Slingshot" }

RULE HookshotEnablesCliffAccess:
  WHEN AcquireItem("Hookshot")
  THEN ReachArea("CliffLedge") WITHIN 10 STEPS

RULE OcarinaBeforePastTemple:
  WHEN FirstEnterNode("TempleOfTime_Past")
  REQUIRE HasItem("Ocarina")

RULE FireTunicBeforeCraterDepths:
  WHEN EnterRoom("DeathMountainCrater_Deep")
  REQUIRE HasItem("FireTunic")

RULE BowIsUsedInForestArc:
  WHEN AcquireItem("Bow")
  THEN CompletePlot("ForestArc") WITHIN 100 STEPS

RULE NeverEnterWaterTempleWithoutIronBoots:
  NEVER (EnterRoom("WaterTemple") AND NOT HasItem("IronBoots"))
```

5 Grammar (Minimal)

A compact grammar sufficient for early prototypes:

$$\begin{aligned}
Spec &::= Clause^+ \\
Clause &::= \text{RULE } Name : Body \\
Body &::= \text{WHEN } Event \text{ THEN } Obligation \\
&\quad | \text{WHEN } Event \text{ REQUIRE } Pred \\
&\quad | \text{NEVER } Pred \\
Event &::= \text{EnterRoom}(Str) \\
&\quad | \text{EnterPlot}(Str) \\
&\quad | \text{FirstEnterNode}(Str) \\
&\quad | \text{AcquireItem}(Str) \\
Obligation &::= \text{ReachObject}(Str) \text{ WITHIN } k \text{ STEPS} \\
&\quad | \text{ReachArea}(Str) \text{ WITHIN } k \text{ STEPS} \\
&\quad | \text{CompletePlot}(Str) \text{ WITHIN } k \text{ STEPS} \\
Pred &::= Atom \mid \text{NOT } Pred \mid Pred \text{ AND } Pred \mid Pred \text{ OR } Pred \mid (Pred) \\
Atom &::= \text{HasItem}(Str) \mid \text{HasAnyItem}\{StrList\} \\
&\quad | \text{HasSkill}(Str) \mid \text{HasAnySkill}\{StrList\} \\
&\quad | \text{HasSeenNode}(Str) \mid \text{AtNode}(Str)
\end{aligned}$$

6 Property Patterns and Compilation

We compile the DSL into a small catalog of temporal-property templates. Let:

- ev be an *event proposition* true on transitions where the triggering event occurs (e.g., `enterRoom.InnUpperFloor`)
- p, q be state predicates.

6.1 Triggered Precondition

DSL form.

$$\text{WHEN } E \text{ REQUIRE } P$$

Compilation (weak, state-at-trigger).

$$G(ev_E \rightarrow p_P)$$

This reads: whenever event E occurs, predicate P must hold in the resulting state.

Optional strong precedence. To enforce “ P must have occurred before the *first* E ”, we can encode:

$$\neg ev_E \text{ } U \text{ } p_P$$

or equivalently restrict the transition relation so that E is disabled until P holds.

6.2 Triggered Bounded Reachability

DSL form.

$$\text{WHEN } E \text{ THEN ReachObject}(O) \text{ WITHIN } k \text{ STEPS}$$

Two compilation strategies are practical:

1. **Derived predicate approach (recommended).** Define a derived state predicate $\text{reach}_O^{\leq k}$ that is true iff the context-specific navigation/action graph reachable from the post-event state has a path of length $\leq k$ to O . Then compile to:

$$G(\text{ev}_E \rightarrow \text{reach}_O^{\leq k})$$

2. **Bounded eventuality approach.** If “STEPS” refers to discrete transitions in the unified model:

$$G(\text{ev}_E \rightarrow F^{\leq k} \text{reach}_O)$$

6.3 Safety (Forbidden Condition)

DSL form.

$$\text{NEVER } P$$

Compilation.

$$G(\neg p_P)$$

6.4 Response Pattern (Optional Extension)

Although not shown in the minimal grammar, many design intents fit:

$$\text{IF } P \text{ THEN EVENTUALLY } Q \rightsquigarrow G(p \rightarrow F q)$$

This is useful for ensuring that acquiring a major item is actually used in a plot arc.

7 Worked Compilations

BedReachableInRoomR

```
WHEN EnterRoom("InnUpperFloor")
THEN ReachObject("Bed") WITHIN 20 STEPS
```

Let enterRoom_Inn be the event proposition, and $\text{reach}_{\text{Bed}}^{\leq 20}$ the derived predicate.

$$G(\text{enterRoom_Inn} \rightarrow \text{reach}_{\text{Bed}}^{\leq 20})$$

TutorialBeforeBoss

```
WHEN FirstEnterNode("Boss_Guardian")
REQUIRE HasSeenNode("StatusEffectTutorial")
```

Let firstEnter_Boss be the event proposition and seen_StatusTut the state predicate.

$$G(\text{firstEnter_Boss} \rightarrow \text{seen_StatusTut})$$

SkillAvailableForPlotX

```
WHEN EnterPlot("Plot_X")
REQUIRE HasAnySkill { "Parry", "DodgeRoll" }
```

Let enterPlot_X be the event proposition.

$$G(\text{enterPlot_X} \rightarrow (\text{hasSkill_Parry} \vee \text{hasSkill_DodgeRoll}))$$

8 Bounded Reachability as an Analysis Side-Condition

To keep the temporal layer simple and efficient, we recommend treating bounded reachability obligations as derived predicates.

Definition

Given a story state s with an associated navigation/action graph $G_s = (V, E)$, a start vertex v_0 (e.g., the player’s spawn location in the room), and a target labelled by object O , define:

$$\text{reach}_O^{\leq k}(s) = \begin{cases} \text{true} & \text{if } \exists \text{ a path of length } \leq k \text{ from } v_0 \text{ to } O \\ \text{false} & \text{otherwise.} \end{cases}$$

Implementation Sketch

- On each transition that emits an event E referenced by a reachability rule, extract the post-event state s' and obtain the context graph $G_{s'}$.
- Run a bounded BFS (or Dijkstra if weighted steps) up to depth k .
- Set the atomic proposition $\text{reach}_O^{\leq k}$ for that state.
- The model checker verifies the simple LTL implication $G(\text{ev}_E \rightarrow \text{reach}_O^{\leq k})$.

This splits concerns cleanly: the story-graph model checker reasons over branching narrative structure, while the navigation check remains a local, bounded graph computation.

9 Discussion: Existential vs. Universal Readings

Many game-design intents are *existential*: “there exists a player path to the bed within 20 steps.” The derived predicate approach naturally captures this by checking for the existence of a short path in the navigation graph.

If a rule must guarantee reachability under all player choices (a stronger and less common requirement), the underlying navigation/action model must encode player choice explicitly; the temporal property then shifts toward universal controllability, which may require game-specific abstractions.

10 Summary

We presented a tiny story-spec DSL inspired by property-pattern approaches in runtime verification and CPS testing. The DSL provides designer-friendly templates that compile to LTL/MTL-style formulas over an abstract state comprising quest nodes and resources, with optional bounded reachability obligations over context-specific navigation graphs. This combination supports practical, scalable checks of narrative ordering constraints, capability gates, and local accessibility guarantees typical of Zelda-like progression.